FLIGHT BOOKING SYSTEM USING DOUBLY LINKED LIST

1. Problem Statement

To develop a Flight Booking System using a Doubly Linked List in C++. The system stores passenger booking information such as Booking ID, Passenger Name, Source, and Destination. It allows users to book flights, display booking records in forward and backward directions, search for bookings, and cancel bookings dynamically using linked list operations. The project also demonstrates pointers and dynamic memory allocation functions such as `malloc()`, `calloc()`, `realloc()`, and `free()`.

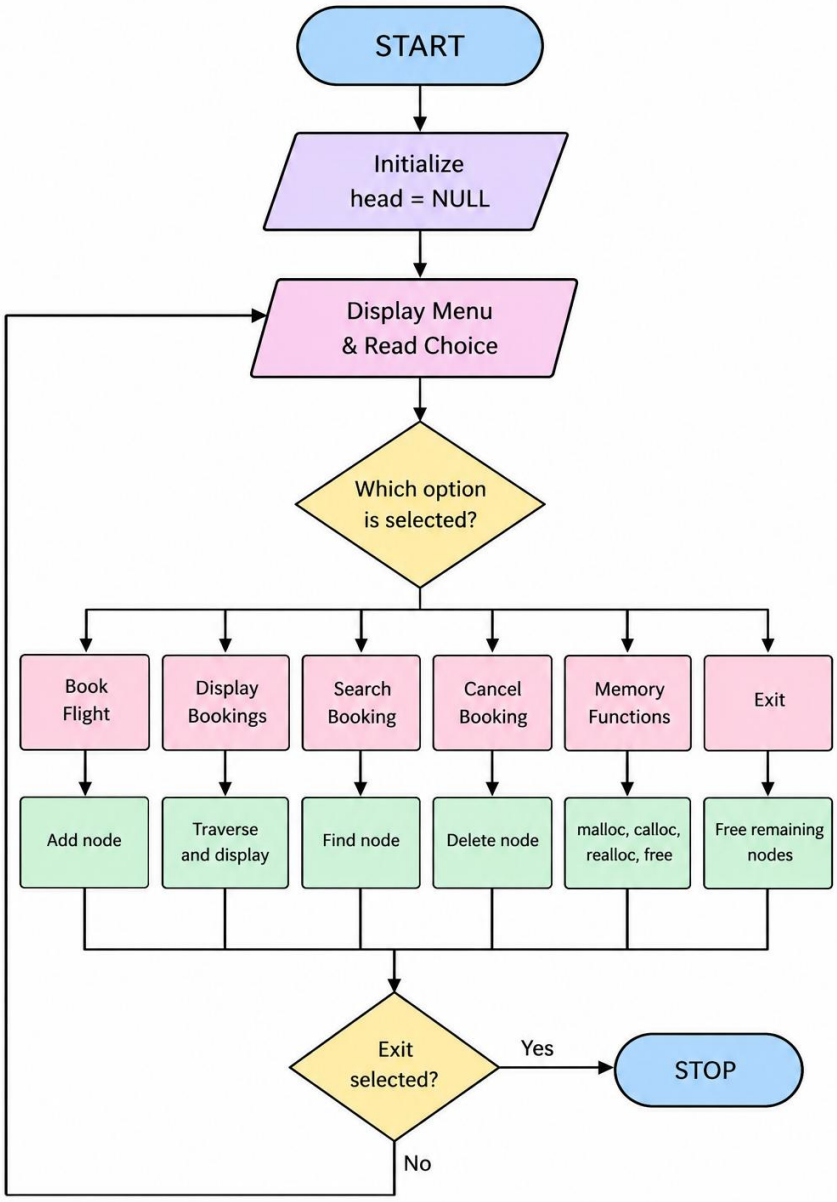
2. Objectives

- To understand the concept and implementation of a Doubly Linked List.
- To implement pointers and dynamic memory allocation in C++.
- To create and store flight booking records dynamically.
- To perform insertion, searching, deletion, and traversal operations.
- To display booking records in both forward and backward directions.
- To understand the use of `malloc()`, `calloc()`, `realloc()`, and `free()`.
- To understand linked organization and its advantages over sequential organization.
- To improve programming, logical thinking, and problem-solving skills.

3. Algorithm

1. Start the program.
2. Define the `Booking` node with booking details and `prev` and `next` pointers.
3. Initialize `head = NULL`.
4. Display the menu and accept the user's choice.
5. Create a new node dynamically to book a flight.
6. Display bookings in forward or backward order.
7. Search for a booking using its Booking ID.
8. Cancel a booking by updating pointers and deleting the node.
9. Demonstrate `malloc()`, `calloc()`, `realloc()`, and `free()`.
10. Repeat until the user selects Exit.
11. Free the remaining nodes and stop the program.

4. Flowchart



5. Conclusion

The Flight Booking System using a Doubly Linked List successfully demonstrates the practical implementation of linked list concepts in C++. It provides a dynamic method for storing, searching, displaying, and cancelling flight bookings. The project helps in understanding pointers, nodes, forward and backward traversal, insertion, deletion, and dynamic memory allocation. It also demonstrates `malloc()`, `calloc()`, `realloc()`, and `free()`, improving practical programming and problem-solving skills.